Simulating hematopoietic phylogeny for SCM validation

Michael Tassia

2026-07-31

Overview

The objective of this notebook is to document our validation strategy for applying stochastic character mapping as a technique for reconstructing somatic genotypes given a phylogenetic tree, substitution model, and a data set of observed genotype states at the tips (see *Bollback 2006* and *Revell 2024*). As a ground-truth somatic phylogeny data set does not yet exist, we will leverage simulation as a framework for validation.

This notebook will first describe the approach, key functions, and expected outcomes. Afterwards, additional layers of technical and biological variation will be introduced, including:

- Sample size
- Sequencing depth
- Missing data
- Collapsing branch lengths equal to 0
- ISM-violation rate

Simulating hematopoiesis

We use the publicly available `rsimpop` package developed by Nick Williams at the Sanger Institute to simulate hematopoiesis under a birth-death model. This package implements the Gillepsie algorithm (see here for basic overview) to simulate cellular compartments under a distribution of user-defined parameters and is described in the supplementary information of *Mitchell et al. 2022*.

Here, we use `rsimpop` with parameters set following empirical estimates from the literature. Fitness coefficients manifest as increased symmetric division rates, and polygenic effects are assumed to be additive.

The libraries and functions used in this notebook are tracked in `simphy_functions.R`.

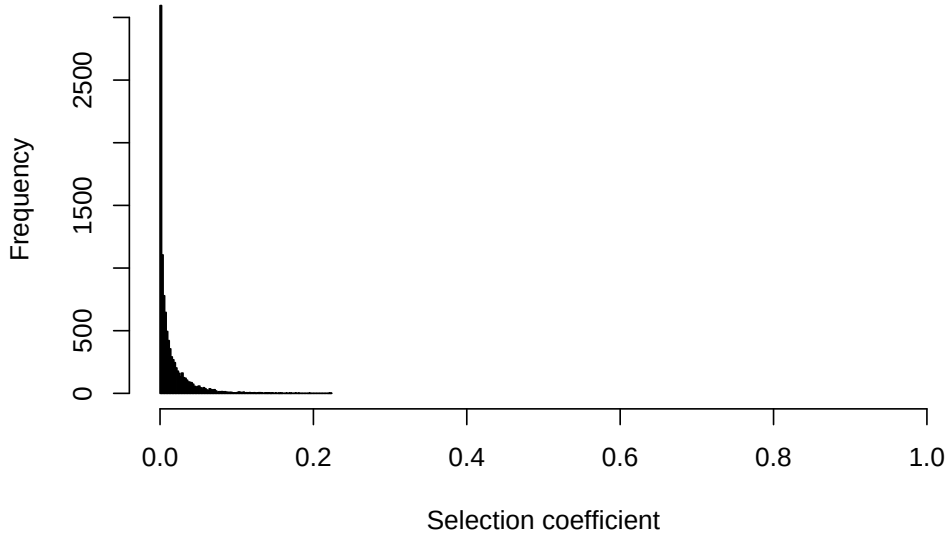
```
suppressMessages(
  source("simphy_functions.R")
)
```

Distribution of fitness effects (DFE)

`rsimpop` encodes fitness as increased rates of symmetric division (i.e., birth) for clones bearing a driver mutation. Here, we draw fitness effects from a gamma distribution using the parameters described in Extended Data Fig 12 of *Mitchell et al. 2022*.

```
SEED <- 1212121

# DFE function
fitnessFn <- genGammaFitness(shape = 0.47,
```

```
                              rate = 34,
                              seed = SEED)

# Plot DFE
hist(sapply(1:1e4, function(x) exp(fitnessFn()) - 1),
     breaks = 100, xlim = c(0, 1),
     xlab = "Selection coefficient", main = "")
```



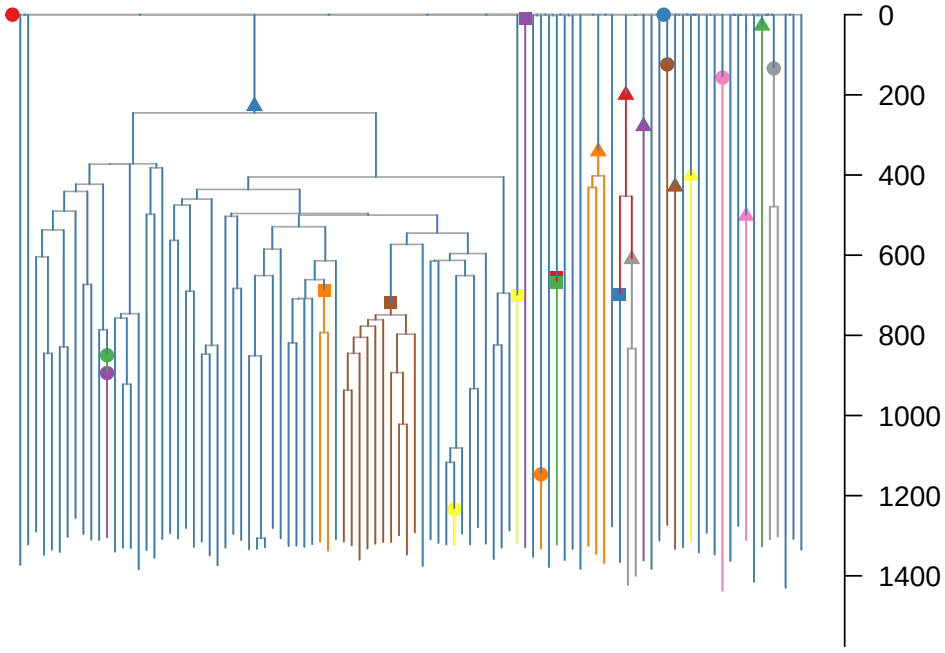
Simulate HSC population

Using the DFE above, we can simulate hematopoiesis with stochastic introduction of driver mutations using the `run_driver_process_sim()` wrapper in `rsimpop`. This function has several important parameters which will be defined below:

- `simpop = NULL`: Default setting; do not use a pre-existing simpop object
- `initial_division_rate = 0.1`: Default setting; parameter estimate for HSCs during development; only effects early population growth phase.
- `final_division_rate = 1/365`: Default setting; parameter estimate for HSCs during adulthood (concordant with the ABC estimates in *Mitchell et al. 2022*).
- `target_pop_size = 1e5`: Default setting; mean parameter estimate for HSC population size at adulthood (concordant with the ABC estimates in *Mitchell et al. 2022*).
- `nyears = 50`: User-defined age at collecting data. Validation will integrate across a distribution of ages.
- `fitness = fitnessFn`: User-defined fitness function defined above
- `drivers_per_cell_per_day = 2.0e-3 / 365`: User-defined count of driver mutations entering the population per day. This is the mean value estimated by ABC in *Mitchell et al. 2022*.

```
# Simulation seed
initSimPop(SEED, bForce = TRUE)

# Run rsimpop
sim <- run_driver_process_sim(
  simpop                  = NULL,
  initial_division_rate   = 0.1,
```

```
  final_division_rate      = 1/365,
  target_pop_size          = 1e5,
  nyears                   = 80,
  fitness                  = fitnessFn,
  drivers_per_cell_per_day = 2.0e-3 / 365
)

# Sample 100 cells from the HSC population
st <- get_subsampled_tree(sim, 100)

# Re-scale the phylogeny by mutations
st_mut <- get_elapsed_time_tree(st,
                                mutrateperdivision = 0,
                                backgroundrate = 16.8/365)

# Plot driver events on the phylogeny
plot_tree_events(st_mut,
                 cex.label = 0,
                 legpos = NULL)
```

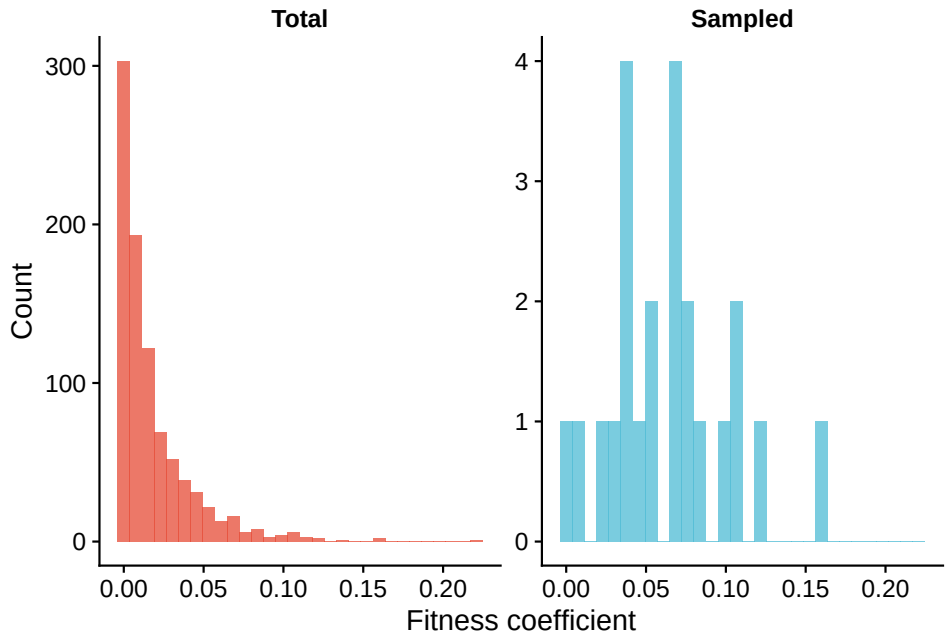


Last before creating a mutation matrix that'll ultimately be used to approximate a VCF-like data structure, we need to extract the `phylo` object from the `simpop` output. The `get_phylo_object()` function also propagates the driver information. As such, we'll plot the driver events drawn for the full population, and overlay indicators to reflect the fitness coefficients sampled in the subset of 100 cells drawn above.

```
# Remove s1 from phylogeny
tr <- get_phylo_object(st_mut)

# Plot fitness distribution
sim$cfg$drivers %>%
  mutate(pop = "Total") %>%
  bind_rows(st_mut$cfg$drivers %>%
              mutate(pop = "Sampled")) %>%
```

```
  mutate(pop = factor(pop, levels = c("Total", "Sampled"))) %>%
  ggplot(aes(x = fitness, fill = pop)) +
  geom_histogram(alpha = 0.75,
                 position = "identity") +
  scale_fill_npg(guide = "none") +
  facet_wrap(~pop, ncol = 2, scales = "free_y") +
  theme_cowplot() +
  theme(legend.position = "top",
        strip.background = element_blank(),
        strip.text = element_text(face = "bold")) +
  labs(x = "Fitness coefficient",
       y = "Count")
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



To compare this ground-truth tree (where branch lengths are scaled by the true mutation counts) against a molecular phylogeny, we need to rescale these raw counts by the size of the analyzable genome. Scaling will also account for the reduced genome access, as only genomic positions passing a genome-accessibility filter are the only loci realistically analyzed during phylogenetic inference (we use the 1000 Genomes Project genome-accessibility mask).

Note that the branch lengths produced using the methods above will be orders of magnitude smaller than the branch lengths fit by maximum-likelihood methods (e.g., `CellPhy`), as the latter are scaled to the number of sites in the alignment (not the whole genome).

```
# Path to a downloaded 1000 Genomes accessibility mask (e.g. the "strict" or
# "pilot" whole-genome mask BED/BED.gz from the URL above)
mask_path <- "https://ftp.1000genomes.ebi.ac.uk/vol1/ftp/data_collections/1000_genomes_project/working/

# Ground-truth molecular phylogeny, branch lengths in substitutions/site
tr_mol <- scale_branches_to_subs_per_site(tr, mask = mask_path)
```

Simulate mutation data

Because `rsimpop` tracks mutation origin alongside the "ground truth" simulated history, we can use these mutations to reconstruct a VCF-like structure that can later be used for SCM.

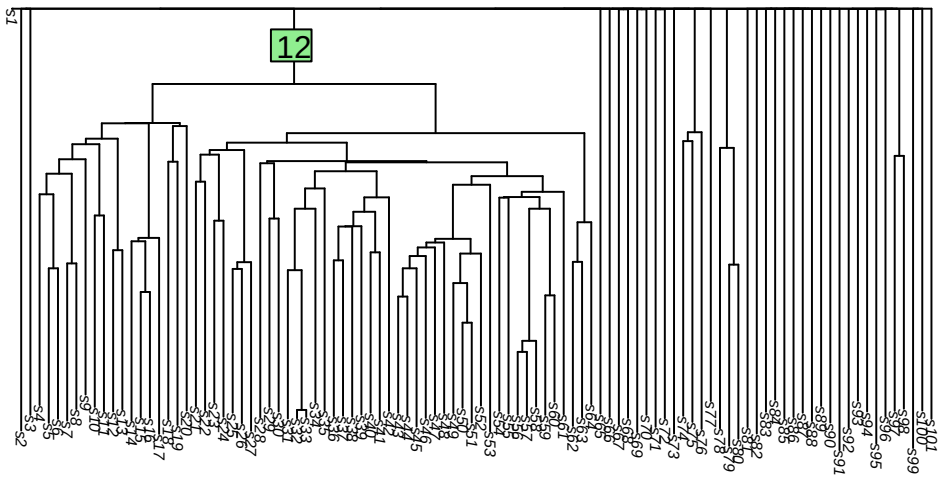
```
# Extract phyletic patterns per mutation
mut_mat <- create_mut_df(tr)

# Preview mutation matrix
mut_mat %>%
  filter(is_driver) %>%
  select(1:9) %>%
  head()
```

```
   mutation_id is_driver selection_coefficient edge s1 s2 s3 s4 s5
1 9eab5e106b97      TRUE           0.156144292   12  0  0  0  1  1
2 ff5d4ba6c61c      TRUE           0.002890073   33  0  0  0  0  0
3 68428b295a3b      TRUE           0.037374682   33  0  0  0  0  0
4 146bbd275beb      TRUE           0.004329532   87  0  0  0  0  0
5 2aff399e0e13      TRUE           0.071059874   93  0  0  0  0  0
6 57744887c3ef      TRUE           0.022098242  122  0  0  0  0  0
```

The `G` object above is a dataframe that contains both meta data (`mutation_id`, `is_driver`, `selection_coefficient`, and `edge`). as well as a numeric boolean array of 0s and 1s, where rows correspond to mutations and columns correspond to sampled cells. A value of 1 indicates that the mutation is present in the cell, while a value of 0 indicates that it is absent.

For example:

```
# Plot tree with edge labels
plot.phylo(tr,
           show.tip.label = TRUE,
           direction = "downwards",
           cex = 0.5)
edgelabels(edge = 12)
```




```
# For the first row of mut_mat where mut_mat$edge == 1, print columns where mutation is present
ingroup <- mut_mat %>%
    filter(edge == 12) %>%
    head(n = 1) %>%
    dplyr::select(!1:4) %>%
    as.vector() == 1
```

```
ingroup <- names(ingroup[ingroup])

cat(paste("Mutation events on edge 12 are inherited by the following tips:\n",
          ingroup[1],
          "to",
          ingroup[length(ingroup)]))
```

```
Mutation events on edge 12 are inherited by the following tips:
 s4 to s64
```

From this, we can use the `simulate_depth_and_ad` function to simulate sequencing data. Default parameters are set such that they mimic the structure of clean data described in both *Mitchell et al. 2022* and Coorens et al. 2025.

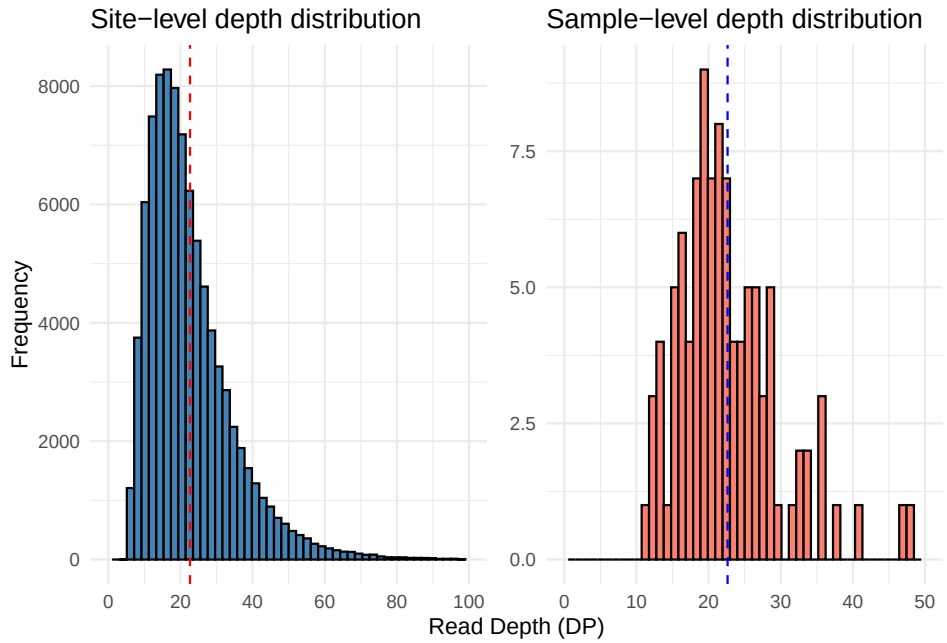
- `G`: Matrix output by `create_mut_df()`
- `D_bar = 20`: Target mean sequencing depth across the whole matrix
- `min_cov = 6`: Lower depth bound a site must clear to pass QC (Sequoia-style min_cov)
- `max_cov = 250`: Upper depth bound (excludes repeat/mapping-artifact loci)
- `site_sdlog = 0.5`: Log-scale spread of per-site coverage bias (GC/mappability)
- `sample_sdlog = 0.3`: Log-scale spread of per-sample library depth variation
- `theta = 8`: Negative-binomial dispersion for residual (post-QC) depth noise
- `rho_max = 0.1`: Beta-binomial overdispersion ceiling a site must clear (Sequoia snv_rho)
- `site_conc_shape = 4`: Gamma shape for the distribution of per-site VAF concentration
- `site_conc_rate = 0.3`: Gamma rate for the distribution of per-site VAF concentration
- `error_rate = 0.002`: Sequencing/mapping error rate (alt reads at true hom-ref sites)
- `dropout_rate = 0.00`: P(complete allelic dropout|truly heterozygous site)

```
sim_reads <- simulate_DP_and_AD(G = mut_mat,
                                seed = SEED)
glimpse(sim_reads)
```

```
List of 7
 $ DP           : num [1:89503, 1:101] 37 14 14 35 31 6 36 8 16 14 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
 $ AD           : int [1:89503, 1:101] 0 0 0 0 0 0 0 0 0 0 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
 $ site_factor  : num [1:89503] 1.363 1.048 0.531 2.209 1.331 ...
 $ sample_factor: num [1:101] 1.119 1.15 0.851 0.907 0.893 ...
 $ site_conc    : num [1:89503] 13.1 26.6 15.6 10.9 14.4 ...
 $ site_rho     : num [1:89503] 0.0708 0.0362 0.0603 0.0843 0.0649 ...
 $ dropout      : logi [1:89503, 1:101] FALSE FALSE FALSE FALSE FALSE FALSE ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
```

The `simulate_DP_and_AD()` function includes multiple stochastic elements that simulate site factors and sample factors that affect read depth. Below, we'll visualize the realized distribution of mean read depth across all sites and samples.

```
Attaching package: 'patchwork'

The following object is masked from 'package:cowplot':

    align_plots
```



From the `sim_reads` list output, the `AD` and `DP` data frames contain the simulated read data at each site for each sample. Using these values, we can approximate Phred-scaled genotype likelihoods following a structure similar to the fixed-error-rate GATK model (DePristo et al. 2011).

```
sim_genotypes <- simulate_GQ_PL_GT(AD = sim_reads$AD,
                                   DP = sim_reads$DP)
glimpse(sim_genotypes)
```

```
List of 5
 $ GQ : num [1:89503, 1:101] 99 42 42 99 93 18 99 24 48 42 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
 $ PL0: num [1:89503, 1:101] 0 0 0 0 0 0 0 0 0 0 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
 $ PL1: num [1:89503, 1:101] 111 42 42 105 93 18 108 24 48 42 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
 $ PL2: num [1:89503, 1:101] 255 255 255 255 255 255 255 255 255 255 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
```

```
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
 $ GT : chr [1:89503, 1:101] "0/0" "0/0" "0/0" "0/0" ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:101] "s1" "s2" "s3" "s4" ...
```

Finally, we will sample sites from the human genome and assign these to the records held in `sim_genotypes`. These sites will then be threaded into a function to create a VCF-like structure.

```
mut_table <- sample_mutation_loci(nrow(sim_genotypes$GT),
                                  seed = SEED)
vcf_df <- build_vcf_df(G = mut_mat,
                       DP = sim_reads$DP,
                       AD = sim_reads$AD,
                       gl = sim_genotypes,
                       edges = mut_mat$edge)

head(vcf_df[, 1:11])
```

```
  CHROM      POS           ID REF ALT QUAL FILTER
1 chr12 15544219 0d388541cac6   C   T    .   PASS
2  chr8 82690969 400a698fbf46   G   A    .   PASS
3  chr2 52646319 9cd57f9f05f7   A   T    .   PASS
4 chr20 52746467 ae1cac52c1b2   A   C    .   PASS
5 chr12 89201926 075e14019b8d   A   C    .   PASS
6  chr3  6357529 3936f67f9c77   A   G    .   PASS
                                    INFO         FORMAT
1 AC=1;AF=0.0049505;EDGE=10;DRIVER=0;S=0 GT:DP:AD:GQ:PL
2 AC=1;AF=0.0049505;EDGE=10;DRIVER=0;S=0 GT:DP:AD:GQ:PL
3 AC=1;AF=0.0049505;EDGE=10;DRIVER=0;S=0 GT:DP:AD:GQ:PL
4 AC=1;AF=0.0049505;EDGE=10;DRIVER=0;S=0 GT:DP:AD:GQ:PL
5 AC=1;AF=0.0049505;EDGE=10;DRIVER=0;S=0 GT:DP:AD:GQ:PL
6 AC=1;AF=0.0049505;EDGE=10;DRIVER=0;S=0 GT:DP:AD:GQ:PL
                        s1                         s2
1 0/0:37:37,0:99:0,111,255  0/1:18:5,13:99:255,0,255
2  0/0:14:14,0:42:0,42,255  0/1:20:4,16:99:255,0,255
3  0/0:14:14,0:42:0,42,255   0/1:10:7,3:99:255,0,255
4 0/0:35:35,0:99:0,105,255 0/1:75:36,39:99:255,0,255
5  0/0:31:31,0:93:0,93,255 0/1:36:21,15:99:255,0,255
6    0/0:6:6,0:18:0,18,255  0/1:13:10,3:99:255,0,255
```

Note that because some samples will have zero coverage at a locus, this can result in some true variants being unobserved in the simulated VCF structure. For instance:

```
vcf_df %>%
  filter(grepl(x = INFO, pattern = "AC=0")) %>%
  dplyr::select(1:8) %>%
  head(n = 5)
```

```
  CHROM      POS           ID REF ALT QUAL FILTER
1 chr13 33645441 4357d1f67086   G   C    .   PASS
2  chrX 78555312 777c0ba79ed2   C   G    .   PASS
3 chr15 65459218 4e23fdf8ed4a   C   A    .   PASS
4  chr3 53168563 66e97d0a67c2   G   C    .   PASS
5 chr12 18026175 c9dd576b57a9   A   T    .   PASS
                           INFO
1 AC=0;AF=0;EDGE=10;DRIVER=0;S=0
```

```
2 AC=0;AF=0;EDGE=10;DRIVER=0;S=0
3 AC=0;AF=0;EDGE=10;DRIVER=0;S=0
4 AC=0;AF=0;EDGE=10;DRIVER=0;S=0
5 AC=0;AF=0;EDGE=10;DRIVER=0;S=0
```

The VCF-like data frame can be exported as a VCF file with `write_vcf_df()`; we will write the ground-truth molecular phylogeny to file alongside it.

```
write_vcf_df(vcf_df %>% filter(!grepl(x = INFO,
                                      pattern = "AC=0")),
             file = "test_sim/test_sim.vcf",
             tree = tr)
write.tree(tr_mol, file = "test_sim/test_sim.nwk")
```

Testing the SCM framework on simulated data

Given the simulated tree and somatic variants above, we can now perform SCM and infer the evolutionary history of somatic variants. Here, we'll use the ground truth tree and just test the performance of SCM in reproducing each variant's origin on the tree. In practice, SCM would be performed on a molecular phylogeny inferred from the VCF file above.

The three critical components for modeling somatic SNVs with SCM are (1) the phylogeny with branch lengths measured in substitutions per site, (2) the VCF file with somatic SNVs and their associated confidence metrics (`PL` scores), and (3) a genotype substitution model fit to the phylogeny in (1). Below, we'll use a substitution model fit with `CellPhy` to the ground truth tree.

The `CellPhy` command used to fit model parameters was the following:

```
cellphy.sh \
    RAXML \
    --evaluate \
    --msa test_sim/test_sim.nwk \
    --msa-format vcf \
    --model GT10+FO \
    --tree test_sim/test_sim.vcf \
    --prefix cellphy \
    --threads 8 \
    --force perf_threads
```

As the function above does produce a phylogeny with branch lengths adjusted for the fit substitution model parameters, a small comparison will be performed below between the ground truth tree and the `bestTree` output by `CellPhy`. The tree output by CellPhy will be used later, as the branch lengths are scaled to the substitution model fit.

```
# read tree
cellphy_tr <- read.tree("test_sim/cellphy.raxml.bestTree")

# Robinson-Foulds distance
RobinsonFoulds(tr_mol, cellphy_tr, normalize = TRUE)
```

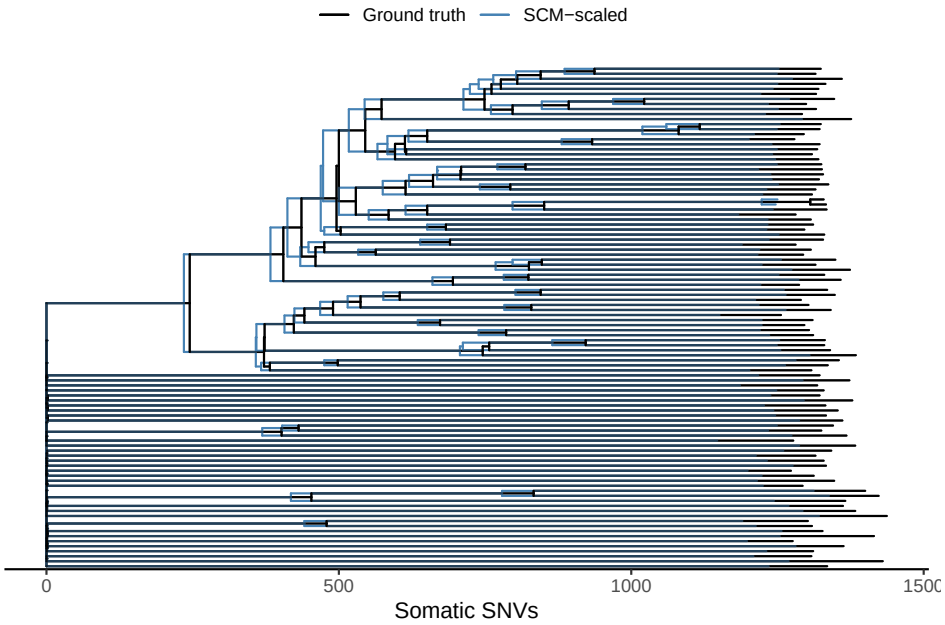
```
[1] 0
```

Validating SCM output with "ground-truth" tree

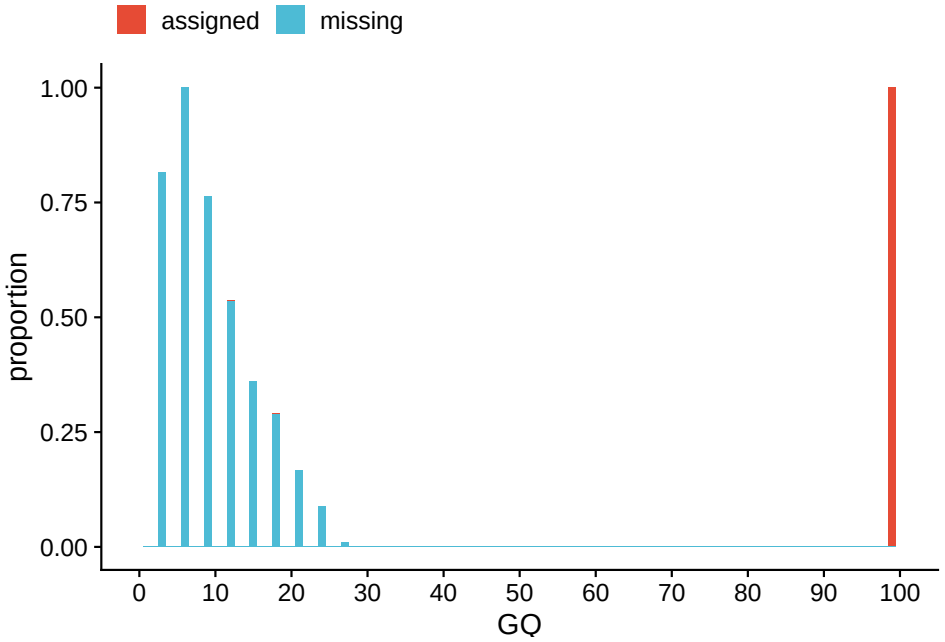
In preparation for applying SCM, we'll load all the functions below:

```
suppressPackageStartupMessages(
  source("../../../smk/workflow/bin/SCM.smk.R")
)
```

The SCM Snakemake pipeline was run on the simulated data above with 100 SCM replicates per mutation. For qualitative insight, we will plot the SCM-scaled tree alongside the ground truth tree.



In the figure above, we see that the SCM-scaled tree is conservative in placing mutations to edges relative to ground truth. Before doing a quantitative comparison of SCM performance per-mutation, below we will compare genotype quality scores for variants that are not assigned to the tree. Below is a histogram showing the difference in genotype quality scores between variants assigned and unassigned to the tree. This quality, while limiting the reproducibility of the ground-truth tree, is nonetheless valuable for accuracy.

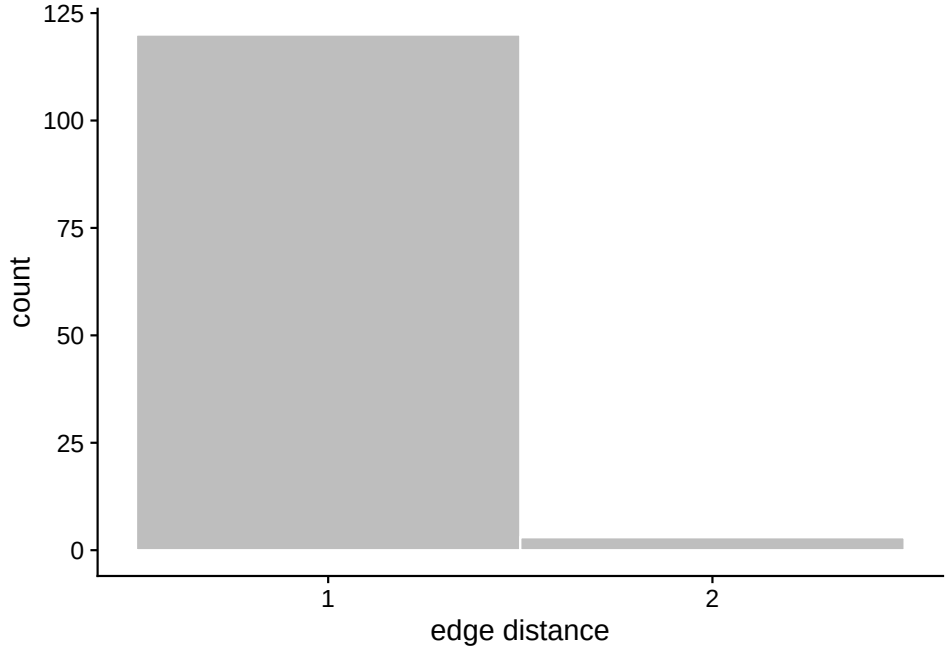


Next, we can ask about concordance between the SCM-inferred origins for the mutations and compare to

ground truth:

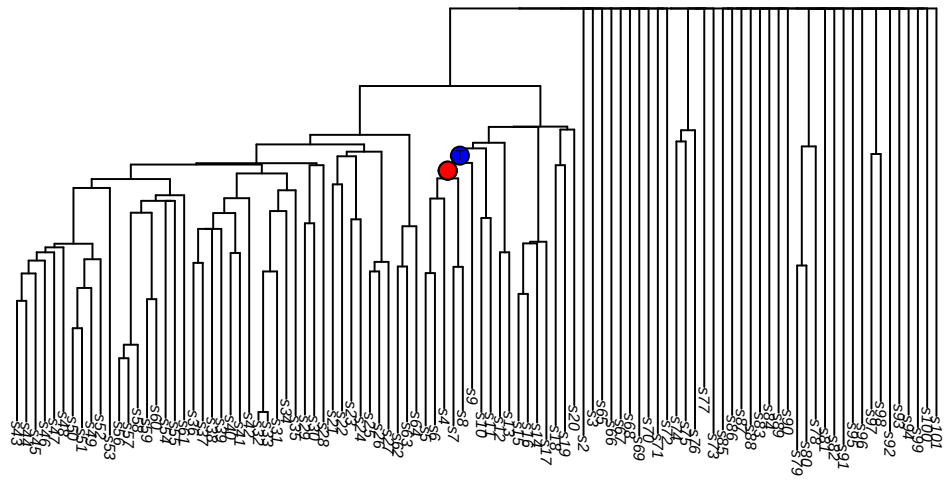
```
[1] "Concordance between SCM-inferred and ground truth mutation origins: 0.999"
```

Below, we plot a histogram of the distance between ground truth and SCM-inferred edges for all mismapped mutations.



The major error modality here is low depth at a variant locus. For example, the variant at `chr10:7337951` is truly present in s4|s5|s6|s7|s8|s9. Upon examination, however, these are the VCF records for these samples:

```
# A tibble: 6 x 6
  sample GT    DP    AD    GQ    PL
  <chr>  <chr> <chr> <chr> <chr> <chr>
1 s4     0/1   4     2,2   99    255,0,255
2 s5     0/0   4     4,0   12    0,12,255
3 s6     0/1   16    8,8   99    255,0,255
4 s7     0/1   12    4,8   99    255,0,255
5 s8     0/1   5     1,4   99    255,0,255
6 s9     0/0   8     8,0   24    0,24,255
```



Validating SCM output with "collapsed" tree

Instead of using the simulated "ground-truth", we can also apply the SCM to a "collapsed" tree, where edges with length 0 are removed to create polytomies.